

Hadoop According to new TextBook.pptx

Exported: May 28, 2026

Summary

This textbook excerpt provides a comprehensive overview of **Hadoop**, an open-source framework specifically designed to manage the **massive scale and variety of Big Data** that modern organizations generate every second. To address the limitations of traditional databases, Hadoop utilizes a **distributed master-slave architecture** composed of two primary layers: the **Hadoop Distributed File System (HDFS)** for reliable, fault-tolerant storage and **MapReduce** for parallel data processing. The text details the evolution from Hadoop 1.0 to **Hadoop 2.0 (YARN)**, explaining how the newer system improves cluster efficiency by separating **resource management** from the application life cycle. Additionally, the material covers practical operational aspects, such as **rack-aware replication** to prevent data loss and various **scheduling algorithms** like FIFO, Fair, and Capacity schedulers that dictate how computational tasks are prioritized. Ultimately, the source serves as a technical guide to how Hadoop enables **batch processing** of gigantic datasets by spreading workloads across clusters of **commodity hardware**.

Keywords: Hadoop Core Components, HDFS Architecture, MapReduce Programming Model, YARN Resource Management, Hadoop Scheduling Algorithms

Content

Hadoop – An Introduction
Big Data and Analytics by Seema Acharya and Subhashini Chellappan
Copyright 2015, WILEY INDIA PVT. LTD.
Big Data and Analytics by Seema Acharya and Subhashini Chellappan
Copyright 2015, WILEY INDIA PVT. LTD.
Hadoop
Ever wondered why Hadoop has been and is one of the most wanted technologies!!
The key consideration (the rationale behind its huge popularity) is: Its capability to handle massive amounts of data, different categories of data – fairly quickly.
The other considerations are : RDBMS versus HADOOP
Big Data and Analytics by Seema Acharya and Subhashini Chellappan
Copyright 2015, WILEY INDIA PVT.

LTD.RDBMS versus HADOOPBig Data and Analytics by Seema Acharya and Subhashini ChellappanCopyright 2015, WILEY INDIA PVT. LTD.Distributed Computing ChallengesBig Data and Analytics by Seema Acharya and Subhashini ChellappanCopyright 2015, WILEY INDIA PVT. LTD.Distributed Computing ChallengesBig Data and Analytics by Seema Acharya and Subhashini ChellappanCopyright 2015, WILEY INDIA PVT. LTD.Hardware FailureHow to Process This Gigantic Store of Data?Hadoop OverviewBig Data and Analytics by Seema Acharya and Subhashini ChellappanCopyright 2015, WILEY INDIA PVT. LTD.Key Aspects of HadoopBig Data and Analytics by Seema Acharya and Subhashini ChellappanCopyright 2015, WILEY INDIA PVT. LTD.Hadoop ComponentsBig Data and Analytics by Seema Acharya and Subhashini ChellappanCopyright 2015, WILEY INDIA PVT. LTD.Hadoop ComponentsBig Data and Analytics by Seema Acharya and Subhashini ChellappanCopyright 2015, WILEY INDIA PVT. LTD.Hadoop Core Components:HDFS:Storage component.Distributes data across several nodes.Natively redundant.MapReduce:Computational framework.Splits a task across multiple nodes.Processes data in parallel.Support projects to enhance the functionality of Hadoop Core ComponentsHivePigScoopHbaseFlumeOozieMahoutHadoop EcosystemHadoop Conceptual LayerHadoop is conceptually divided intoData storage layer which stores huge volume of data and Data processing layer which processes data in parallel to extract richer and meaningful insights from dataHadoop High Level ArchitectureBig Data and Analytics by Seema Acharya and Subhashini ChellappanCopyright 2015, WILEY INDIA PVT. LTD.Hadoop is a distributed master slave architecture.Master node- Name nodeSlave node- Data nodeMaster HDFS- Its Main responsibility is partitioning data storage across the slave node. It also keeps track of location of data on data nodesMaster Mapreduce- It decides and schedules computation task on slave nodeHadoop DistributorsBig Data and Analytics by Seema Acharya and Subhashini ChellappanCopyright 2015, WILEY INDIA PVT. LTD.Hadoop DistributorsBig Data and Analytics by Seema Acharya and Subhashini ChellappanCopyright 2015, WILEY INDIA PVT. LTD.HDFS(HADOOP DISTRIBUTED FILE SYSTEM)Big Data and Analytics by Seema Acharya and Subhashini ChellappanCopyright 2015, WILEY INDIA PVT. LTD.HDFS is a distributed file system (DFS) that runs on large clusters and provides high-throughput access to data. HDFS is a highly fault-tolerant system and is designed to work with commodity hardware. HDFS stores each file as a sequence of blocks. The blocks of each file are replicated on multiple machines in a cluster to provide fault toleranceLet us look at the characteristics of HDFS:• Scalable Storage for Large Files: HDFS has been designed to store large files . Large files are broken into chunks or blocks and each block is replicated across multiple machines in the cluster. HDFS has been designed to scale to clusters comprising of thousands of nodes.• Replication: HDFS replicates data blocks to multiple machines in a cluster which makes the system reliable and fault-tolerant. The default block size used is

64MB and the default replication factor is 3.

- **Streaming Data Access:** HDFS has been designed for streaming data access patterns and provides high throughput streaming reads and writes. The HDFS make it suitable for batch operations thus trading off interactive access capability. This design choice has been made to meet the requirements of applications that involve write-once, read many times data access patterns.
- **File Appends:** Recent versions of HDFS have introduced the append capability.

HDFS Architecture Figure shows the architecture of HDFS. HDFS has two types of nodes: Namenode and Datanode.

Namenode Namenode manages the filesystem namespace. All the filesystem meta-data is stored on the Namenode. While Namenode is responsible for executing operations such as opening and closing of files, no data actually flows through the Namenode. Namenode executes the read and write operations while the data is transferred directly to/from the Datanodes. HDFS splits files into blocks, and the blocks are stored on the Datanodes. For each block, multiple replicas are kept. Namenode persistently stores the filesystem meta-data and the mappings of the blocks to the datanodes, on the disk as two files: fsimage and edits files. .The fsimage contains a complete snapshot of the filesystem meta-data. The edits file stores the incremental updates to the meta-data. When the Namenode starts, it loads the fsimage file into the memory and applies the edits file to bring the in-memory view of the filesystem up-to-date. Namenode then writes a new fsimage file to the disk.

Secondary Namenode The edits file keeps growing in size, over time, as the incremental updates are stored. The responsibility of applying the updates to the fsimage file is delegated to the Secondary Namenode, as the Namenode may not have enough resources available, as it is performing other operations. This process is called checkpointing. The checkpointing process is done either periodically (default 1 hour) or after a certain number of unchecked transactions have been reached on the Namenode. The new fsimage is uploaded by the Secondary Namenode to the Namenode. When the checkpointing process begins, the Secondary Namenode downloads the fsimage and edits files from the Namenode to the checkpoint directory on the Secondary Namenode. The Secondary Namenode then applies the edits on the fsimage file and creates a new fsimage file.

Datanode While the Namenode stores the filesystem meta-data, the Datanodes store the data blocks and serve the read and write requests. Datanodes periodically send heartbeat messages and block reports to the Namenode. While the heartbeat messages tell the Namenode that a Datanode is alive, the block reports contain information on the blocks on a Datanode.

Data Blocks & Replication Blocks are replicated on the Datanodes and by default three replicas are created. The placement of replicas on the Datanodes is determined by a rack-aware placement policy. This placement policy ensures reliability and availability of the blocks. For a replication factor of three, one replica is placed on a node on a local rack, the second replica is placed on a different node on a remote rack and the third replica is placed on a different node on the same remote rack. This ensures

that even if the rack becomes unavailable, at least one replica will remain available. Placement of replicas on different nodes in the same rack minimizes the network traffic between the racks. HDFS Read Path Figure shows the HDFS read path. The read process begins with the client sending a request to the Namenode to obtain the locations of the data blocks for a file. The Namenode checks if the file exists and whether the client has sufficient permissions to read the file. The Namenode responds with the data block locations sorted by the distance to the client. This helps in minimizing the traffic between the nodes as the client can read the blocks from the nearest node. For example, if the client is on the same node as a data block, it can read the data block locally. The client reads the data blocks directly from the Datanodes in order, till all the blocks have been read. The Datanodes stream the data to the client. During the read process, if a replica becomes unavailable, the client can read another replica on a different Datanode. HDFS Write Path Figure shows the HDFS write path. The write process begins with the client sending a request to the Namenode to create a new file in the filesystem namespace. The Namenode checks if the user has sufficient permissions to create the file and whether the file doesn't already exist in the filesystem. The Namenode responds to the client with an output stream object. The client writes data to the output stream object which splits the data into packets and enqueues them into a data queue. The packets are consumed from the data queue in a separate thread, which requests the Namenode to allocate new blocks on the Datanodes to which the data should be written. Namenode responds with the locations of the blocks on the Datanodes. The client then establishes direct connections to the Datanodes on which the blocks are to be replicated forming a replication pipeline. The data packets consumed from the data queue are written to the first Datanode on the replication pipeline, which writes data to the second Datanode in the pipeline and so on. Once the packets are successfully written, each Datanode in the pipeline sends an acknowledgment. The client keeps a track of which all packets are acknowledged by the Datanodes. The process of writing data packets to the Datanodes proceeds till the block size is reached. Upon reaching the block size, the client again requests the Namenode to return a set of new blocks on the Datanodes. The client then streams the packets to the Datanodes. This process repeats till all the data packets are written and acknowledged. Finally, the client closes the output stream and sends a request to the Namenode to close the file.

HDFS Commands Summary

HDFS is a distributed file system that runs on large clusters and provides high-throughput access to data. HDFS provides scalable storage for large files which are broken into blocks. The blocks are replicated to make the system reliable and fault-tolerant. The HDFS Namenode stores the filesystem meta-data and is responsible for executing operations such as opening and closing of files. The Secondary Namenode helps in the checkpointing process by applying the updates in the edits file to the fsimage file which contains a complete snapshot of the filesystem meta-data. Datanodes

store the data blocks which are replicated. The placement of replicas on the Datanodes is determined by a rack-aware placement policy. We described examples of accessing HDFS using the command line tools, a Python library for HDFS and the HDFS web interface.

Hadoop and MapReduce Apache Hadoop is an open source framework for distributed batch processing of big data. Similarly, MapReduce is a parallel programming model i.e, a software framework suitable for analysis of big data. MapReduce algorithms allow large-scale computations to be automatically parallelized across a large cluster of servers.

MapReduce Programming Model MapReduce is a parallel data processing model for processing and analysis of massive scale data . MapReduce model has two phases: Map and Reduce. MapReduce programs are written in a functional programming style to create Map and Reduce functions. The input data to the map and reduce phases is in the form of key-value pairs. Run-time systems for MapReduce are typically large clusters built of commodity hardware. The MapReduce run-time systems take care of tasks such partitioning the data, scheduling of jobs and communication between nodes in the cluster. This makes it easier for programmers to analyze massive scale data without worrying about tasks such as data partitioning and scheduling.

In the Map phase, data is read from a distributed file system, partitioned among a set of computing nodes in the cluster, and sent to the nodes as a set of key-value pairs. The Map tasks process the input records independently of each other and produce intermediate results as key-value pairs. The intermediate results are stored on the local disk of the node running the Map task. When all the Map tasks are completed, the Reduce phase begins in which the intermediate data with the same key is aggregated. An optional Combine task can be used to perform data aggregation on the intermediate data of the same key for the output of the mapper before transferring the output to the Reduce task.

It is only suitable for Batch Processing of Huge amount of Data. It is not suitable for Real-time Data Processing. It is not suitable for Data Streaming. It supports upto 4000 Nodes per Cluster. It has a single component : JobTracker to perform many activities like Resource Management, Job Scheduling, Job Monitoring, Re-scheduling Jobs etc. JobTracker is the single point of failure. It supports only one Name Node and One Namespace per Cluster. It runs only Map/Reduce jobs.

Drawback of Hadoop 1.0

Hadoop 2 YARN- Taking Hadoop beyond Batch

Hadoop YARN is the next generation architecture of Hadoop (version 2.x). In the YARN architecture, the original processing engine of Hadoop (MapReduce) has been separated from the resource management component (which is now part of YARN) as shown Figure shows the MapReduce job execution workflow for the next generation Hadoop MapReduce framework (MR2). The next-generation MapReduce architecture divides the two major functions of the JobTracker in Hadoop 1.x – resource management and job life-cycle management – into separate components – ResourceManager and ApplicationMaster. The key components of YARN are described as follows:

- Resource Manager (RM): RM manages the global assignment

of compute resources to applications. RM consists of two main services:- Scheduler: Scheduler is a pluggable service that manages and enforces the resource scheduling policy in the cluster.- Applications Manager (AsM): AsM manages the running Application Masters in the cluster. AsM is responsible for starting application masters and for monitoring and restarting them on different nodes in case of failures.Application Master (AM): A per-application AM manages the application's life cycle. AM is responsible for negotiating resources from the RM and working with the NMs to execute and monitor the tasks.Node Manager (NM): A per-machine NM manages the user processes on that machine.Containers: Container is a bundle of resources allocated by RM (memory, CPU and network). A container is a conceptual entity that grants an application the privilege to use a certain amount of resources on a given machine to run a task.Each node has an NM that spawns multiple containers based on the resource allocations made by the RM Figure 7.4 shows a YARN cluster with a Resource Manager node and three Node Manager nodes. There are as many Application Masters running as there are applications (jobs). Each application's AM manages the application tasks such as starting, monitoring and restarting tasks in case of failures. Each application has multiple tasks. Each task runs in a separate container. Containers in YARN architecture are similar to task slots in Hadoop MapReduce 1.x (MR1). However, unlike MR1 which differentiates between map and reduce slots, each container in YARN can be used for both map and reduce tasks. The resource allocation model in MR1 consists of a predefined number of map slots and reduce slots. This static allocation of slots results in low cluster utilization. The resource allocation model of YARN is more flexible with the introduction of resource containers which improve cluster utilization.To better understand the YARN job execution workflow let us analyze the interactions between the main components on YARN. Figure 7.5 shows the interactions between a Client and Resource Manager. Job execution begins with the submission of a new application request by the client to the RM. The RM then responds with a unique application ID and information about cluster resource capabilities that the client will need in requesting resources for running the application's AM.Using the information received from the RM, the client constructs and submits an Application Submission Context which contains information such as scheduler queue, priority and user information. The Application Submission Context also contains a Container Launch Context which contains the application's jar, job files, security tokens and any resource requirements. The client can query the RM for application reports.The client can also "force kill" an application by sending a request to the RMFigure 7.6 shows the interactions between Resource Manager and Application Master. Upon receiving an application submission context from a client, the RM finds an available container meeting the resource requirements for running the AM for the application. On finding a suitable container, the RM contacts the NM for the container to start the AM process on its node. When the AM is launched it registers itself with the RM. The

registration process consists of handshaking that conveys information such as the RPC port that the AM will be listening on, the tracking URL for monitoring the application's status and progress, etc. The AM relays heartbeat and progress information to the RM. The AM sends resource allocation requests to the RM that contains a list of requested containers, and may also contain a list of released containers by the AM. Upon receiving the allocation request, the scheduler component of the RM computes a list of containers that satisfy the request and sends back an allocation response. Upon receiving the resource list, the AM contacts the associated NMs for starting the containers. When the job finishes, the AM sends a Finish Application message to the RM. The registration response from the RM contains information for the AM that is used in calculating and requesting any resource requests for the application's individual tasks (such as minimum and maximum resource capabilities for the cluster). Figure 7.7 shows the interactions between the an Application Master and the Node Manager. Based on the resource list received from the RM, the AM requests the hosting NM for each container to start the container. The AM can request and receive a container status report from the Node Manager. Figure 7.8 shows the MapReduce job execution within a YARN cluster.

Hadoop Schedulers

The scheduler is a pluggable component in Hadoop that allows it to support different scheduling algorithms. The pluggable scheduler framework provides the flexibility to support a variety of workloads with varying priority and performance constraints. The Hadoop scheduling algorithms are described as follows:

FIFO

FIFO scheduler maintains a work queue in which the jobs are queued. The scheduler pulls jobs in first-in first-out manner (oldest job first) for scheduling. There is no concept of priority or size of the job in FIFO scheduler.

Fair Scheduler

The Fair Scheduler allocates resources evenly between multiple jobs and also provides capacity guarantees. Fair Scheduler assigns resources to jobs such that each job gets an equal share of the available resources on average over time. Fair Scheduler lets short jobs finish in reasonable time while not starving long jobs. Tasks slots that are free are assigned to the new jobs, so that each job gets roughly the same amount of CPU time. The Fair Scheduler maintains a set of pools into which jobs are placed. Each pool has a guaranteed capacity. When there are multiple jobs in the pools, each pool gets at least as many task slots as guaranteed. Each pool receives at least the minimum share. When a pool does not require the guaranteed share the excess capacity is split between other jobs. This lets the scheduler guarantee capacity for pools while utilizing resources efficiently when these pools don't contain jobs. The Fair Scheduler keeps track of the compute time received by each job. The scheduler computes periodically the difference between the computing time received by each job and the time it should have received in ideal scheduling. The job which has the highest deficit of the compute time received is scheduled next. This ensures that over time, each job gets its fair share of compute time. Fair scheduler is useful when a small or large Hadoop cluster is

shared between multiple groups of users in an organization. Though the fair scheduler ensures fairness by maintaining a set of pools and providing guaranteed capacity to each pool, it does not provide any timing guarantees and hence it is ill-equipped for real-time jobs.

Capacity Scheduler. In Capacity Scheduler, multiple named queues are defined, each with a configurable number of map and reduce slots. Each queue is also assigned a guaranteed capacity. The Capacity Scheduler gives each queue its capacity when it contains jobs, and shares any unused capacity between the queues. Within each queue FIFO scheduling with priority is used. For fairness, it is possible to place a limit on the percentage of running tasks per user, so that users share a cluster equally. A wait time for each queue can be configured. When a queue is not scheduled for more than the wait time, it can preempt tasks of other queues to get its fair share. When a TaskTracker has free slots, the Capacity Scheduler picks a queue for which the ratio of number of running slots to capacity is the lowest. The scheduler then picks a job from the selected queue to run. Jobs are sorted based on when they're submitted and their priorities. Jobs are considered in order, and a job is selected if its user is within the user-quota for the queue, i.e., the user is not already using queue resources above the defined limit. The capacity scheduler is useful when a large Hadoop cluster is shared between with multiple clients and different types and priorities of jobs. Though the capacity scheduler ensures fairness by maintaining a set of queues and providing guaranteed capacity to each queue, it does not provide any timing guarantees and, therefore, it may be ill-equipped for real-time jobs.

Word Count: Map step: mapper.py Reduce step: reducer.py

Reference: Writing An Hadoop MapReduce Program In Python (michael-noll.com)

Hadoop - MapReduce Examples

Batch Analysis of Sensor Data Figure 7.9 shows a Hadoop MapReduce workflow for batch analysis of weather data. Batch analysis is done to aggregate data (such as computing mean, maximum, and minimum) on various timescales. For this example, we will assume that we have a data collector which retrieves the sensor data collected in the cloud database and creates a raw data file in a form suitable for processing by Hadoop. The raw data file consists of the raw sensor readings along with the timestamps as shown below:

"2015-04-29 10:15:32",38,42,34,5 : "2015-04-30 10:15:32",87,48,21,4

Box 7.1 shows the map program for the batch analysis of sensor data. The map program reads the data from standard input (stdin) and splits the data into the timestamp and individual sensor readings. The map program emits key-value pairs where the key is a portion of the timestamp (that depends on the timescale on which the data is to be aggregated), and the value is a comma separated string of sensor readings

Box 7.1: Map program - mapper.py

```
#!/usr/bin/env python
import sys

# Calculates mean temperature, humidity, light and CO2
# Input data format: "2014-04-29 10:15:32",37,44,31,6
# Output: "2014-04-29 10:15 [48.75, 31.25, 29.0, 16.5]"
# Input comes from STDIN (standard input)
for line in sys.stdin:
    # remove leading and trailing whitespace
    line = line.strip()
    data = line.split(',')
    l=len(data)
    # For
```



```
aggregation          by          minutekey=str(data[0]  
[0:17])value=data[1]+' '+data[2]+' '+data[3]+' '+data[4]print '%s /div>
```